Software Requirement Specifications

Image Recognition and Detection Application

Submitted by

Amna Javad

S25BINFT7M04014

Submitted to

Mam Sara Fareed

Department of Information Technology
Faculty of Computing
The Islamia University of Bahawalpur

Meeting Details

Sr No	Details	Date	Supervisor Signature
1			
2			
3			
4			
5			
6			
7			
8			
9			
10			

Summary

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for the Image Recognition and Detection Application — a final year project developed at the Department of Information Technology, Faculty of Computing, The Islamia University of Bahawalpur.

The proposed system leverages deep learning techniques, specifically Convolutional Neural Networks (CNNs), to identify and detect objects, faces, text, and scenes in images captured via a mobile or web camera. The application is designed to assist users in real-time image analysis, providing accurate classification results with bounding box annotations. This document outlines the project scope, user characteristics, functional and non-functional requirements, use cases, and proposed technologies to guide the development team through the entire software development lifecycle.

Table of Contents

1. Introduction 4
1.1. Purpose 4
1.2. Scope 4
1.3. Product Perspective 4
1.4. User Characteristics 5
1.5. Similar Apps and Systems / Literature Review 5
1.6. Proposed Technologies 5
2. Requirements 6
2.1. Functional Requirements 6
2.2. Non-Functional Requirements 9
3. Use Cases and Flow of Processes 10
3.1. Use Case 1 – Upload Image for Recognition 10
3.2. Use Case 2 – Real-Time Detection via Camera 11
3.3. Use Case 3 – View Detection History 11
4. References 12

1. Introduction

The Image Recognition and Detection Application is a standalone AI-powered system developed as a Final Year Project at the Islamia University of Bahawalpur. The project addresses the growing need for intelligent visual processing tools in everyday life — from automated surveillance to accessibility aids for visually impaired users. The application uses state-of-the-art deep learning models to perform real-time detection and classification of objects, faces, and textual content within images. Users can upload images or use a live camera feed to receive instant AI-generated analysis with annotated output.

1.1. Purpose

The purpose of this SRS is to formally document the requirements for the Image Recognition and Detection Application. This document serves as a reference for the development team, the project supervisor, and evaluators. It defines what the system shall do, the constraints under which it shall operate, and the expected quality attributes. The goals of this project are to:

- Build a robust image classification and object detection engine using pre-trained deep learning models (e.g., YOLO, MobileNet).
- Provide a user-friendly mobile/web interface for uploading images and viewing detection results.
- Enable real-time inference via device camera with on-screen bounding box overlays.
- Store detection history for registered users to review past results.
- Support multiple detection categories including objects, human faces, animals, vehicles, and printed text (OCR).

1.2. Scope

The Image Recognition and Detection Application will be developed as a web-based system with an optional Progressive Web App (PWA) interface. The scope of development includes the following:

In Scope:

- User registration, login, and profile management.
- Image upload and camera-based live detection.
- Object detection with bounding boxes and confidence scores.
- Image classification into predefined categories.
- Optical Character Recognition (OCR) for text extraction from images.
- Detection history dashboard for registered users.
- Admin panel for managing users and reviewing system logs.

Out of Scope:

- Video streaming or real-time video object tracking.
- Integration with third-party surveillance hardware.
- Custom model training interface for end users.
- Multi-language support (only English UI in version 1.0).

1.3. Product Perspective

The Image Recognition and Detection Application functions as a standalone system consisting of two main components:

- Frontend (Client Side): A React.js-based web interface accessible via any modern browser. The interface allows users to upload images, view live camera feeds, and display detection results overlaid on the image.
- Backend (Server Side): A Python Flask/FastAPI server that receives image data, processes it through deep learning inference pipelines (YOLO v8 for detection, MobileNetV3 for classification), and returns structured JSON results to the frontend.

The system connects to a PostgreSQL database for storing user profiles and detection history. Pre-trained AI models are loaded at server startup and remain in memory for low-latency inference. The application does not depend on any external organizational system and is designed as a self-contained academic prototype.

1.4. User Characteristics

The following user roles interact with the system:

User Role	Description	Technical Expertise
General User	Registers and logs in to upload images or use the camera for object detection. Views results and history.	Low to Moderate (basic smartphone/browser usage)
Guest User	Uses the system without registration. Limited to single-image upload with no history saved.	Low
Administrator	Manages user accounts, reviews system logs, and monitors model performance via admin dashboard.	High (IT background required)

1.5. Similar Apps and Systems / Literature Review

Several existing tools and research works relate to this project:

- Google Lens: A commercial product by Google that allows real-time object recognition via smartphone camera. It identifies products, landmarks, plants, animals, and text. Limitation: requires an active internet connection and does not expose raw detection data to users.
- Microsoft Azure Computer Vision API: A cloud-based API providing object detection, OCR, and face analysis. Limitation: paid service; not suitable for offline or resource-constrained environments.
- YOLO (You Only Look Once) Framework: An open-source, real-time object detection algorithm. Research by Redmon et al. (2016) demonstrated single-pass neural network inference achieving high speed with competitive accuracy. This project builds upon YOLOv8.
- TensorFlow Lite Object Detection: Google's on-device ML solution. Offers lightweight models for mobile deployment but requires advanced setup. This project draws inspiration from its real-time inference design.

- Amazon Rekognition: AWS's image and video analysis service. Feature-rich but vendor-locked and expensive. This project provides a locally-hosted open-source alternative.

1.6. Proposed Technologies

Category	Technology	Purpose
Frontend	React.js, Tailwind CSS	User interface and responsive design
Backend	Python, FastAPI	REST API server, request handling
AI / ML	YOLOv8, MobileNetV3, Tesseract OCR	Object detection, classification, OCR
Deep Learning	PyTorch, OpenCV	Model inference and image preprocessing
Database	PostgreSQL	User data and detection history storage
Authentication	JWT (JSON Web Tokens)	Secure user session management
Deployment	Docker, Nginx	Containerization and production serving
Version Control	Git, GitHub	Source code management

2. Requirements

The Image Recognition and Detection Application must handle the following core operations: user account management (registration, login, profile update), image upload and camera-based capture, real-time inference using pre-trained deep learning models to detect and classify objects with bounding boxes and confidence scores, OCR-based text extraction from images, persistent storage of detection history linked to user accounts, and an administrative backend for monitoring system usage. The system must be accessible via any modern web browser and provide responsive feedback within acceptable latency bounds.

2.1. Functional Requirements

2.1.1. User Registration (Sign Up)

Field	Details
Name	FR001
Purpose	Allows a new user to create an account to access the system's full features.
User(s)	General User
Input	Full Name (actual name), Email Address (valid format), Password (min 8 chars, at least 1 number and 1 special character), Profile Picture (optional, JPG/PNG only)
Output	User account created. Confirmation email sent. User redirected to login page.

2.1.2. User Login

Field	Details
Name	FR002
Purpose	Authenticates a registered user and grants access to personalized features.
User(s)	General User, Administrator
Input	Email Address, Password
Output	JWT access token issued. User redirected to dashboard. Failed login shows error message.

2.1.3. Image Upload for Detection

Field	Details
Name	FR003
Purpose	Enables the user to upload a static image file for AI-based object recognition and detection.
User(s)	General User, Guest User

Input	Image file (JPG, PNG, WEBP; max size 10MB)
Output	Annotated image displayed with bounding boxes, detected object labels, and confidence scores in JSON and visual format.

2.1.4. Real-Time Camera Detection

Field	Details
Name	FR004
Purpose	Allows the user to use their device camera for live frame capture and instant object detection.
User(s)	General User
Input	Live camera stream (device webcam or rear camera via browser API)
Output	Real-time annotated video frames with detected objects labeled, displayed on screen.

2.1.5. Text Extraction (OCR)

Field	Details
Name	FR005
Purpose	Extracts readable text from uploaded images containing printed or handwritten content.
User(s)	General User, Guest User
Input	Image containing text (JPG, PNG)
Output	Extracted text displayed in a text box; user may copy to clipboard.

2.1.6. Detection History

Field	Details
Name	FR006
Purpose	Saves and displays a log of all previous detection results for a registered user.
User(s)	General User
Input	User session (JWT token)
Output	List of past detections with thumbnails, detected labels, date/time, and option to re-view or delete.

2.1.7. Admin Panel

Field	Details
Name	FR007
Purpose	Gives the administrator control over user management, system monitoring, and model performance review.
User(s)	Administrator
Input	Admin login credentials
Output	Admin dashboard showing user list, total detections, system uptime, and option to deactivate accounts.

2.2. Non-Functional Requirements

The following non-functional requirements define the quality attributes the system must satisfy:

Category	Requirement ID	Specification
Performance	NFR001	Image detection API must return results within 3 seconds for images up to 10MB under normal server load.
Reliability	NFR002	System uptime must be at least 99% during active academic evaluation periods.
Security	NFR003	All API endpoints must be secured with JWT authentication. Passwords must be hashed using bcrypt.
Usability	NFR004	A new user must be able to perform image detection without prior training within 5 minutes of visiting the app.
Scalability	NFR005	The backend must handle up to 50 concurrent users without degradation in response time.
Maintainability	NFR006	Code must be modular, documented with inline comments, and version-controlled on GitHub.
Compatibility	NFR007	Frontend must be compatible with Chrome, Firefox, Edge, and Safari on both desktop and mobile.
Accuracy	NFR008	Object detection model must achieve a minimum mAP (mean Average Precision) of 70% on the COCO validation dataset.

3. Use Cases and Flow of Processes

The system-level use cases below represent the primary interactions between actors and the Image Recognition and Detection Application. The main actors are the General User, Guest User, and Administrator. Each use case is linked to one or more functional requirements defined in Section 2.

text

```
+-------------------------- SYSTEM BOUNDARY --------------------------+
|                                                                      |
|   +-------------+                +------------------+               |
|   | Registered  |                |    Guest User    |               |
|   |    User     |                |                  |               |
|   +------+------+                +--------+---------+               |
|          |                                 |                        |
|          | uses                            | uses                   |
|          v                                 v                        |
|   +------------------+            +------------------+              |
|   | 1. Sign Up       |            | 2. Real-Time     |              |
|   |                  |            |    Detection     |              |
|   +------------------+            +------------------+              |
|          |                                 ^                        |
|          | uses                            | (also by registered)   |
|          v                                 |                        |
|   +------------------+            +------------------+              |
|   | 3. Face Register |----------> | 4. Face Recognize|              |
|   +------------------+            +------------------+              |
|          |                                 ^                        |
|          |                                 |                        |
|          v                                 |                        |
|   +------------------+            +------------------+              |
|   | 5. OCR           |            | 6. View History  |              |
|   +------------------+            +------------------+              |
|                                                                      |
+----------------------------------------------------------------------+
```

3.1. Use Case 1 – Upload Image for Recognition

Field	Details
ID	UC001
Name	Upload Image for Recognition
Description	This use case describes the process by which a user uploads an image and receives object detection results.
Requirement(s)	FR003
Actor(s)	General User, Guest User
Precondition	User has a valid image file (JPG, PNG, WEBP) on their device.
Postcondition	Detection results are displayed on screen. Results saved to history (registered users only).
Basic Flow	**Basic Flow** 1. User navigates to the Upload page. 2. User selects an image file via the file picker. 3. System validates the file type and size. 4. User clicks 'Detect' button. 5. System sends image to backend inference API. 6. Backend returns JSON with detected objects, bounding box coordinates, and confidence scores. 7. Frontend renders annotated image and displays results list. **Alternative Flow** A1. User drags and drops image directly onto the upload area. **Exceptions** E1. Invalid file format — system shows error message: 'Unsupported file type.' E2. File exceeds 10MB — system shows error: 'File too large. Maximum size is 10MB.' E3. Server timeout — system shows: 'Detection failed. Please try again.'

3.2. Use Case 2 – Real-Time Detection via Camera

Field	Details
ID	UC002
Name	Real-Time Detection via Camera
Description	This use case describes the flow when a registered user activates the live camera feed for real-time object detection.
Requirement(s)	FR004

Actor(s)	General User
Precondition	User is logged in. Device has a camera. Browser camera permissions are granted.
Postcondition	Live annotated camera view is displayed. User may capture a frame and save to history.
Basic Flow	**Basic Flow** 1. User clicks 'Live Detection' from the navigation menu. 2. Browser requests camera permission from the user. 3. Live video stream begins; frames are sent to the backend every 500ms. 4. Backend processes each frame and returns detection results. 5. Frontend overlays bounding boxes and labels on the live feed. 6. User may click 'Capture' to save the current frame and result. **Exceptions** E1. Camera permission denied — system shows: 'Camera access required for live detection.' E2. No camera found — system shows: 'No camera device detected.'

3.3. Use Case 3 – View Detection History

Field	Details
ID	UC003
Name	View Detection History
Description	Registered users can view a chronological list of all past image detection results.
Requirement(s)	FR006
Actor(s)	General User
Precondition	User is authenticated (logged in).
Postcondition	User can view, re-analyze, or delete past detection records.
Basic Flow	**Basic Flow** 1. User clicks 'History' from the dashboard. 2. System fetches detection records from the database for the logged-in user. 3. Records are displayed with thumbnail, date, time, and top detected label. 4. User clicks a record to view full annotated result. 5. User may delete a record using the delete button. **Exceptions** E1. No records found — system displays: 'No detection history yet. Start by uploading an image!'

Activity Diagram for UC001 (ASCII)

```text
  +---------+
  |  START  |
  +----+----+
       |
       v
+------------------+
| User opens app   |
+------------------+
       |
       v
+------------------+
| User taps 'Sign  |
| Up'              |
+------------------+
       |
       v
+------------------+
| System displays  |
| registration form|
+------------------+
       |
       v
+------------------+
| User fills data  |
| & submits        |
+------------------+
       |
       v
+------------------+      No
| System validates |<-----------------+
| inputs           |                  |
+------------------+                  |
       |                              |
       | Yes                          |
       v                              |
+------------------+                  |
```

```
| System creates   |                   |
| account          |                   |
+------------------+                   |
       |                               |
       v                               |
+------------------+                   |
| Send verification|                   |
| email            |                   |
+------------------+                   |
       |                               |
       v                               |
+------------------+                   |
| Auto-login user  |                   |
+------------------+                   |
       |                               |
       v                               |
+------------------+                   |
| Navigate to main |                   |
| screen           |                   |
+------------------+                   |
       |                               |
       v                               |
  +---------+                          |
  |  END    |                          |
  +---------+                          |
                                       |
                     +-----------------+
                     | Highlight error |
                     | fields          |
                     +-----------------+
```

4. References

The following references are cited in IEEE format:

[1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), Las Vegas, NV, 2016, pp. 779-788.

[2] A. G. Howard et al., "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," arXiv preprint arXiv:1704.04861, 2017.

[3] T. Y. Lin et al., "Microsoft COCO: Common Objects in Context," in European Conference on Computer Vision (ECCV), Zurich, 2014, pp. 740-755.

[4] Ultralytics, "YOLOv8 Documentation," [Online]. Available: https://docs.ultralytics.com. [Accessed: 2024].

[5] Google, "Google Lens," [Online]. Available: https://lens.google.com. [Accessed: 2024].

[6] Microsoft, "Azure Computer Vision," [Online]. Available: https://azure.microsoft.com/en-us/services/cognitive-services/computer-vision/. [Accessed: 2024].

[7] R. Smith, "An Overview of the Tesseract OCR Engine," in Proc. 9th Int. Conf. Document Analysis and Recognition, Parana, 2007, pp. 629-633.

[8] IEEE, "IEEE Recommended Practice for Software Requirements Specifications," IEEE Std 830-1998, Jun. 1998.